

Licenciatura de Engenharia Informática

Departamento de Engenharia Informática e de Sistemas

Trabalho Prático

Programação Orientada a Objetos

Meta 2

Realizado por:

João Francisco Gomes Simões – 2022141647

Ricardo Duarte Rodrigues – 2022147797

Conteúdo

Introdução: 2
Classes: 3
Classe Modelo: 3
Classe Iteração: 4
Classe Zona: 5
Classe Sensor: 6
Classe ProcessadorRegras: 7
Classe Regra: 8
Classe Aparelho: 9
Decisões tomadas: 10
Conclusão: 11

Introdução:

- O trabalho consiste em construir um simulador de uma habitação controlada por componentes de domótica interligados entre si, que reagem a propriedades do ambiente e a comandos do utilizador.
- O simulador inclui zonas da habitação, que têm propriedades como temperatura, luz, etc., que são inspecionadas por sensores, que fornecem os valores lidos a regras que são geridas por processadores de regras, que determinam o que fazer em função das leituras dos sensores. Nas zonas existem também aparelhos, que afetam as propriedades da zona onde se encontram e reagem a comandos emitidos pelos processadores de regras.
- O simulador é controlado por comandos escritos pelo utilizador, que permitem criar, modificar e interagir com os componentes da habitação. O simulador também tem a noção de passagem de tempo, que é independente do tempo real e avança por ordem do utilizador.

Classes:

Classe Modelo:

A classe Modelo é responsável por gerir as janelas e todas as iterações realizadas durante a simulação.

```
class Modelo {
public:
    Modelo();
    void run();
private:
    Terminal &terminal;
    Window JanelaGrafica;
    Window JanelaMensagens;
    Window JanelaComandos;
    Iteracao iteracao;
};
```

Figura 1- Classe Modelo

Classe Iteração:

A classe Iteração é a classe que contém a informação da disposição da habitação e zonas. Além disso, tem também métodos responsáveis nas tarefas a efetuar em cada um dos comandos.

```
class Iteracao {
public:
    Iteracao(Terminal &t, Window &g, Window &m, Window &c);

    void mostrarMensagens(const string& mensagem, int cor);
    int getInstante() const;
    //comandos
    void proximoInstante();
    void avancaInstantes(int n);
    void criarHabitacao(int numLinhas, int numColunas);
    void removerHabitacao();
    void criarZona(int linha, int coluna);
    void removerZona(int idZona);
    void listarZonas();
    void listarComponentes(int idZona);
    void listarProps(int idZona);
    void mudarValorProp(int idZona, string nome, float valor);
    void adicionaComponente(int idZona, string componente, string tipo_comando);
    void removeComponente(int idZona, string componente, int idComponente);
    void criaRegra(int idZona, int idProc, string tipo_regra, int idSensor, vector<float> valores);
    void mudaComandoProcessador(int idZona, int idProc, string novo_comando);
    void listarRegras(int idZona, int idProc);
    void removeRegra(int idZona, int idProc, int idRegra);
    void associaProcApar(int idZona, int idProc, int idApar);
    void desassociaProcApar(int idZona, int idProc, int idApar);
    void enviaComandoAparelho(int idZona, int idApar, string novo_comando);
    void psalva();
    void prepoe();
    void prem();
    void plista();
    void execCommand(const string& nomeArquivo);
private:
    Terminal &terminal;
    Window &JanelaGrafica;
    Window &JanelaMensagens;
    Window &JanelaComandos;
    int instantes = 0;
    int proxIdZona = 0;
    map<int, shared_ptr<Window>> zonaWindows;
    vector<vector<unique_ptr<Zona>>> habitacao;
};
```

Figura 2-Classe Iteração

Classe Zona:

A classe Zona permite armazenar em vários vetores os diferentes, sensores, processadores de regras, aparelhos e através de uma coleção “map” guardamos os valores das propriedades presentes em cada zona. Permite manipular todos as coleções de modo a adicionar, remover e quantificar os diversos elementos.

```
class Zona {

public:
    string listarComponentes(int id);
    string listarProps(int id);
    void setValueProp(string _tipo, float _valor);
    void addSensor(string nome);
    void addProcessador(string comando);
    void addAparelho(string tipo);
    void removeSensor(int _id);
    void removeProcessador(int _id);
    void removeAparelho(int _id);
    void alteracoesInstantes();
    Zona(int id, int l, int c);
    ~Zona();
    int getId() const;
    string getPosicao() const;
    int getNumSensores() const;
    int getNumProcessadores() const;
    int getNumAparelhos() const;
    string zonaNumComp() const;
    ProcessadorRegras *procurarProcessador(int id);
    Sensor *procurarSensor(int id);
    Aparelho *procurarAparelho(int id);
private:
    void init_props();
    int zona_id;
    int pos_lin, pos_col;
    int n_sensores, n_processadores, n_aparelhos;
    vector<Sensor*> sensores;
    vector<ProcessadorRegras *> processadores;
    vector<Aparelho *> aparelhos;
    map<string, float *> props;
};
```

Figura 3- Classe Zona

Classe Sensor:

A classe Sensor é uma classe que nos fornece os dados das propriedades presentes na zona. Um sensor lê os valores da propriedade que for igual à string tipo (tipo de sensor).

```
class Sensor{
public:
    Sensor(int id, string tipo, float* valor);
    ~Sensor();
    int getId() const;
    string getTipo() const;
    float getValorLeitura() const;

private:
    int id;
    string tipo;
    float* valor_leitura;
};
```

Figura 4- Classe Sensor

Classe ProcessadorRegras:

É uma classe que contém métodos que permitem definir regras e verificar a veracidade das mesmas, influenciando os dados das propriedades das zonas ao comunicar com os aparelhos.

```
class ProcessadorRegras {
public:
    ProcessadorRegras(int _id, string _comando);
    int getId() const;
    int getNRegras() const;
    void setComando(string _comando);
    string listarRegras();
    void adicionaAparelho(Aparelho* a);
    void procLigaAparelho();
    void procDesligaAparelho();
    void removerAparelho(int id);
    void adicionarRegraIgual_a(Sensor* sensor, float valor_comparacao);
    void adicionarRegraMenor_que(Sensor* sensor, float valor_comparacao);
    void adicionarRegraMaior_que(Sensor* sensor, float valor_comparacao);
    void adicionarRegraEntre(Sensor* sensor, float minimo, float maximo);
    void adicionarRegraFora(Sensor* sensor, float minimo, float maximo);
    int removerRegra(int id);
    bool avaliarRegras();
private:
    int processador_id;
    int n_regras;
    string comando;
    vector<Regra*> regras;
    vector<Aparelho*> aparelhos;
};
```

Figura 5 - Classe ProcessadorRegra

Classe Regra:

É uma classe que permite a criação das várias regras a ser analisadas pelos processadores de regras. Essas regras são avaliadas comparando o valor de um sensor associado com valores pré definidos.

Existem classes derivadas desta classe, que são as seguintes:

- Igual_a;
- Menor_que;
- Maior_que;
- Entre;
- Fora.

```
class Regra {
public:
    Regra(int _id, Sensor* sensor);
    virtual bool avaliarRegra()=0;
    virtual string getNomeRegra()=0;
    float getValorAtual();
    int getIdRegra() const;
    int getIdSensorAss() const;
    string getNomeSensorAss() const;
private:
    int id;
    Sensor* sensor_associado;
};
```

Figura 6 - Classe Regra

Classe Aparelho:

É uma classe que contém métodos de manipulação dos aparelhos.
Existem classes derivadas desta classe, que são as seguintes:

- Aquecedor;
- Aspersor;
- Refrigerador;
- Lâmpada.

```
class Aparelho {
private:
    int aparelho_id;
    bool isligado;
    int temporizador;
public:
    Aparelho(int _id);
    int getId() const;
    virtual string getNome() const = 0;
    bool getEstado() const;
    void setEstado();
    int getTemporizador() const;
    int incTemporizador();
};
```

Figura 7 - Classe Aparelho

Decisões tomadas:

Na implementação visual do projeto, utilizamos a biblioteca fornecida. Por esse motivo, criamos várias "janelas", sendo que aquela localizada no canto superior esquerdo é dedicada à representação gráfica da habitação, auxiliando o utilizador na compreensão da sua distribuição. A janela no canto superior direito é utilizada para visualizar mensagens relacionadas à simulação, exibindo informações importantes. Por último, a janela localizada no final do terminal destina-se a mostrar os momentos da simulação e a permitir a introdução de comandos pelo utilizador.



Figura 8 - Terminal

Conclusão:

Utilizando C++ como linguagem de programação, desenvolvemos um simulador que nos permitiu definir zonas, sensores, processadores de regras e aparelhos, observando como interagem entre si e com o ambiente. O simulador proporcionou a simulação da passagem do tempo e a capacidade de armazenar e recuperar regras personalizadas. A interface do utilizador foi simples, mas eficaz, apresentando uma representação esquemática da habitação e uma área para inserir e visualizar comandos e resultados. Este projeto representou uma oportunidade para aprender e aplicar conceitos e práticas de programação orientada a objetos, utilizando classes e bibliotecas fornecidas nas aulas ou da biblioteca padrão C++. A realização deste trabalho contribuiu para uma compreensão mais aprofundada de conceitos como encapsulamento, herança, polimorfismo e abstração.